Command Line Arguments

A complete C program

```
/* prog.c  */
#include <stdio.h>

void PRINT();
void PRINT_I(int);
void PRINT_F(float,float);

int main() {
        int a=10;
        float x,y;
        scanf("%d %d",&x,&y);
        PRINT();
        PRINT_I(a);
        PRINT_F(x,y);
                return 0;
}
```

```
void PRINT()
{
        printf("Hello Everybody!!!\n");
}
```

```
void PRINT_I(int b)
{
        printf("%d \n",b);
}
```

```
void PRINT_F(float p, float q)
{
        printf("%2.1f %2.2f \n",p,q);
}
```

Compilation and Execution

```
$ cc –Wall prog.c
$
$ ./a.out
2.34
3.45
Hello Everybody!!!
10
2.3 3.45
$
```

A complete C program

```
/* prog.c  */

#include <stdio.h>
void PRINT();
void PRINT_I(int);
void PRINT_F(float,float);

int main(????){
        int a=10;
        float x,y;
        scanf("%d %d",&x,&y);
        PRINT();
        PRINT_I(a);
        PRINT_F(x,y);
                return 0;
}
```

```
void PRINT()
{
        printf("Hello Everybody!!!\n");
}
```

```
void PRINT_I(int b)
{
        printf("%d \n",b);
}
```

```
void PRINT_F(float p, float q)
{
        printf("%2.1f %2.2f \n",p,q);
}
```

Command Line Arguments

- Command line arguments may be passed by specifying them under main( ).

```
int main(int argc, char *argv[ ]);
```

- argc tells us the number of command line arguments
- argv[0] is the first, argv[1] is the second, argv[2] is the third etc.

What are Command line arguments?

- In unix we can type, for example,

```
$ mv myfile1.dat myfile2.dat
```

- The myfile1.dat and myfile2.dat are known as command line arguments
- But how would we write a C program which could listen to command line arguments

Example code to print arguments

```
int main (int argc, char *argv[]){
    int i;
    for (i= 0; i < argc; i++) {
        printf ("Argv[%d] = %s",i, argv[i]);
    }
    return 0;
}
```

```
$ a.exe how many
argc = 3
argv[0] = a.out
argv[1] = how
argv[2] = many
```

Example: Averaging from Command Line

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc,char *argv[]) {
  float sum=0;
  int    i,num;

   num=argc-1;
  for(i=1;i<=num;i++)
    sum+=atof(argv[i]);
  printf("Average=%f \n",sum/(float) num);
}
```

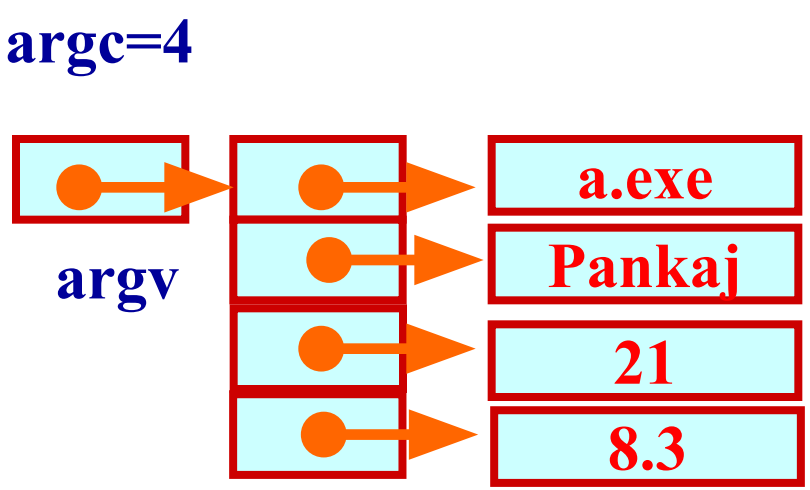
```
$ a.exe 45 239 123
Average=135.666667
```

Passing parameters to main()

```
#include <stdio.h>

int main(int argc, char *argv[])
{
        int i;

        for(i=0;i<argc;i++) {
                printf("%d: %s\n",i,argv[i]);
        }

        return 0;
}
```

```
$ a.exe Anirban 21 8.3
0: ./a.out
1: Pankaj
2: 21
3: 8.3
```

a.exe Pankaj 21 8.3



Library function sscanf()

- Header file:

```
#include <stdio.h>
```

- Function prototype:

```
int sscanf(const char *str, const char *format, ...);
```

- Conversion character is same as scanf().

Passing parameters to main()

```
#include <stdio.h>
int main(int argc, char *argv[])
{
	int i;
	for(i=0;i<argc;i++) {
		printf("%d: %s\n",i,argv[i]);
	}
	return 0;
}
```

```
$ a.exe Pankaj 21 8.3
0: ./a.out
1: Pankaj
2: 21
3: 8.3
$
```

```
#include <stdio.h>
int main(int argc, char *argv[])
{
	char name[20];
	int age;
	float cgpa;

	sscanf(argv[1],"%s",name);
	sscanf(argv[2],"%d",&age);
	sscanf(argv[3],"%f",&cgpa);

	printf("%s  %d   %f\n",
		name,age,cgpa);
	return 0;
}
```

```
$ a.exe Pankaj 21 8.3
Pankaj  21   8.300000
$
```

Functions to convert strings to numbers

- Once we've got a string with a number in it (either from a file or from the user typing) we can use atoi or atof to convert it to a number
- The functions are part of header file `stdlib.h`

```
char numberstring[]= "3.14";
int i;
double pi;
pi= atof (numberstring);
i= atoi ("12");
```

Both of these functions return 0 if they have a problem

Example: Average from Command Line

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc,char *argv[])
{
   float sum=0;
   int    i,num;

   num=argc-1;
   for(i=1;i<=num;i++)
     sum+=atof(argv[i]);
   printf("Average=%f \n",sum/(float) num);
   return 0;
}
```

```
$ a.exe 45  239  123
Average=135.666667
$
```

Passing parameters to main()

- Two parameters will be passed to function main() through command line – argc and argv.
- Name of the parameters are fixed.
- argc is of integer type and it stores the number of parameters (delimited by space) in the command line.
- argv is a 2D array of characters and it stores all the words in the command line.

- By default all the parameters are taken as array of characters (strings) that can be converted to other data types.